# PDF Q&A Bot

Technical Documentation & Architecture Guide

Streamlit • LangChain • OpenAI • FAISS

## 1. Project Overview

PDF Q&A Bot is an interactive web application that allows users to upload one or more PDF documents and ask natural-language questions about their contents. Built with Streamlit for the user interface and powered by the LangChain framework with OpenAI language models, the system performs semantic search over document text and generates contextual, conversational answers.

The application is designed for knowledge workers who need to rapidly query large documents — research papers, contracts, manuals, or reports — without manually reading through them.

**Primary Use Case:** Upload PDFs → Ask questions in plain English → Receive AI-generated answers with source references

## 2. Architecture

The system follows a layered RAG (Retrieval-Augmented Generation) architecture. User documents are ingested, chunked, embedded into a vector store, and queried at runtime to provide grounded answers.

### System Architecture

**USER LAYER** **Web Browser** → Streamlit UI → Upload PDFs + Type Questions

▼ HTTP Requests

| APPLICATION LAYER | PROCESSING LAYER | AI / VECTOR LAYER |
|---|---|---|
| **Streamlit Server** | **Document Pipeline** | **LangChain + OpenAI** |
| Page config & layout | PyPDF2 extraction | OpenAI Embeddings |
| Session state mgmt | File validation (50MB) | FAISS vector store |
| Rate limiter (20/hr) | CharacterTextSplitter | ConversationalRetrieval |
| File uploader UI | chunk=1000, overlap=200 | gpt-4o-mini (temp=0.1) |
| Chat history display | Retry logic (3x) | Top-3 retrieval (k=3) |

▼ API Calls

| OpenAI API | Environment |
|---|---|
| text-embedding-ada-002 \| gpt-4o-mini | .env file \| OPENAI_API_KEY |

# 3. Core Components

## 3.1 User Interface — Streamlit

The frontend is built entirely with Streamlit, providing a browser-based interface without requiring separate HTML/CSS/JS. Key UI components include:

- Sidebar file uploader supporting multiple simultaneous PDF uploads (max 50 MB each)
- Real-time processing status with expandable file details
- Two-column question input with an Ask button
- Expandable chat history showing all prior Q&A pairs in the session
- Source document preview with first 500 characters of retrieved chunks

## 3.2 Document Processing Pipeline

PDF ingestion follows a four-stage pipeline before any question can be answered:

| Stage | Component | Parameters | Output |
|---|---|---|---|
| 1. Validate | validate_file() | Max 50MB, PDF MIME type | Pass/fail + error message |
| 2. Extract | PyPDF2.PdfReader | All pages concatenated | Raw text string |
| 3. Chunk | CharacterTextSplitter | size=1000, overlap=200, sep=\n | List of text chunks |
| 4. Embed | OpenAIEmbeddings + FAISS | text-embedding-ada-002 | FAISS vector store |

## 3.3 QA Chain — LangChain RAG

At question time, the ConversationalRetrievalChain performs the following steps:

- The user's question is embedded using the same OpenAI embedding model
- FAISS performs a cosine similarity search and returns the top-3 (k=3) most relevant text chunks
- The retrieved chunks, the question, and the full chat history are combined into a prompt sent to gpt-4o-mini
- The model returns an answer grounded in the retrieved context, which is appended to session chat history

**Why RAG?** Direct document Q&A without RAG would exceed LLM context windows for large PDFs. RAG retrieves only the relevant sections, keeping costs low and answers precise.

## 4. Security & Reliability

| Feature | Implementation | Configuration |
|---|---|---|
| **API Key Security** | Loaded from .env via python-dotenv | OPENAI_API_KEY environment variable |
| **Rate Limiting** | Per-session counter with hourly reset | 20 questions per hour (configurable) |
| **File Validation** | MIME type check + size enforcement | 50MB max, PDF only |
| **Retry Logic** | Exponential backoff on API failures | 3 attempts, 1s base delay |
| **Error Handling** | Per-file try/catch with logging | Partial failure continues pipeline |
| **Caching** | st.cache_resource on heavy ops | Embeddings + QA chain reused |

## 5. Configuration Reference

All tunable parameters are defined as module-level constants at the top of pdf_bot.py:

| Constant | Default | Description |
|---|---|---|
| MODEL_NAME | gpt-4o-mini | OpenAI chat model used for answer generation |
| CHUNK_SIZE | 1000 | Character count per text chunk |
| CHUNK_OVERLAP | 200 | Overlap between consecutive chunks (preserves context) |
| MAX_FILE_SIZE_MB | 50 | Maximum allowed PDF upload size in megabytes |
| RATE_LIMIT_QUESTIONS | 20 | Maximum questions allowed per user per hour |
| RETRY_ATTEMPTS | 3 | Number of retry attempts on API call failure |
| RETRY_DELAY | 1s | Base delay (seconds) for retry backoff |

## 6. Data Flow

### 6.1 Ingestion Flow (one-time per upload)

1. User uploads PDF(s) via sidebar file uploader
2. Each file is validated (size + MIME type); invalid files are warned and skipped
3. PyPDF2 extracts raw text from all pages of each valid PDF
4. CharacterTextSplitter divides text into overlapping chunks of 1000 characters
5. OpenAI Embeddings converts each chunk into a 1536-dimension vector
6. FAISS indexes all vectors in memory; the docsearch object is cached by Streamlit
7. ConversationalRetrievalChain is instantiated and also cached for reuse

### 6.2 Query Flow (per question)

1. User types a question; rate limiter checks the hourly quota
2. Question text is embedded into a vector using the same OpenAI embedding model
3. FAISS returns the top-3 most semantically similar document chunks
4. LangChain builds a prompt: retrieved chunks + question + full chat history
5. gpt-4o-mini generates a grounded answer (temperature=0.1 for consistency)
6. Answer is appended to st.session_state.chat_history and rendered in the UI
7. Source document snippets are displayed in a collapsible expander

## 7. Dependencies

| Package | Category | Purpose |
|---|---|---|
| streamlit | UI Framework | Web app server, session state, widgets |
| langchain | AI Orchestration | Text splitting, chains, retrieval |
| langchain-openai | AI Orchestration | OpenAI embeddings and chat model wrappers |
| openai | External API | OpenAI API client (gpt-4o-mini, embeddings) |
| faiss-cpu | Vector Store | In-memory similarity search over embeddings |
| PyPDF2 | Document Parsing | PDF text extraction |
| python-dotenv | Configuration | .env file loading for API keys |

## 8. Setup & Deployment

**Prerequisites**

- Python 3.8 or higher

**Installation**

- Clone or download the repository
- Install dependencies: pip install streamlit langchain langchain-openai faiss-cpu PyPDF2 python-dotenv
- Create a .env file in the project root and add: OPENAI_API_KEY=your_api_key
- Run the application: streamlit run pdf_bot.py
- Open the browser URL shown in the terminal (typically http://localhost:8501)

**Note:** The FAISS index is stored entirely in memory. Data is not persisted between application restarts. For production use, consider persisting the FAISS index to disk using faiss.write_index().

## 9. Known Limitations

- Scanned PDFs without embedded text will extract as empty strings — OCR support (e.g., pytesseract) would be needed for scanned documents
- The FAISS vector index is held in memory only; re-uploading the same PDF after a server restart re-runs the full embedding pipeline. For production use, persist the index to disk using faiss.write_index()
- Rate limiting is per-browser-session only and does not aggregate across multiple users or server instances — a shared backend store (e.g., Redis) would be required for multi-user deployments
- The updated code uses LangChain's LCEL-based create_retrieval_chain and create_history_aware_retriever (replacing the deprecated ConversationalRetrievalChain). If downgrading to LangChain < 0.2, revert to the original chain API

## 10. UI Walkthrough

The screenshot below shows the running application after a PDF has been uploaded and a question answered. Each numbered area corresponds to a key UI region described in the annotations that follow.

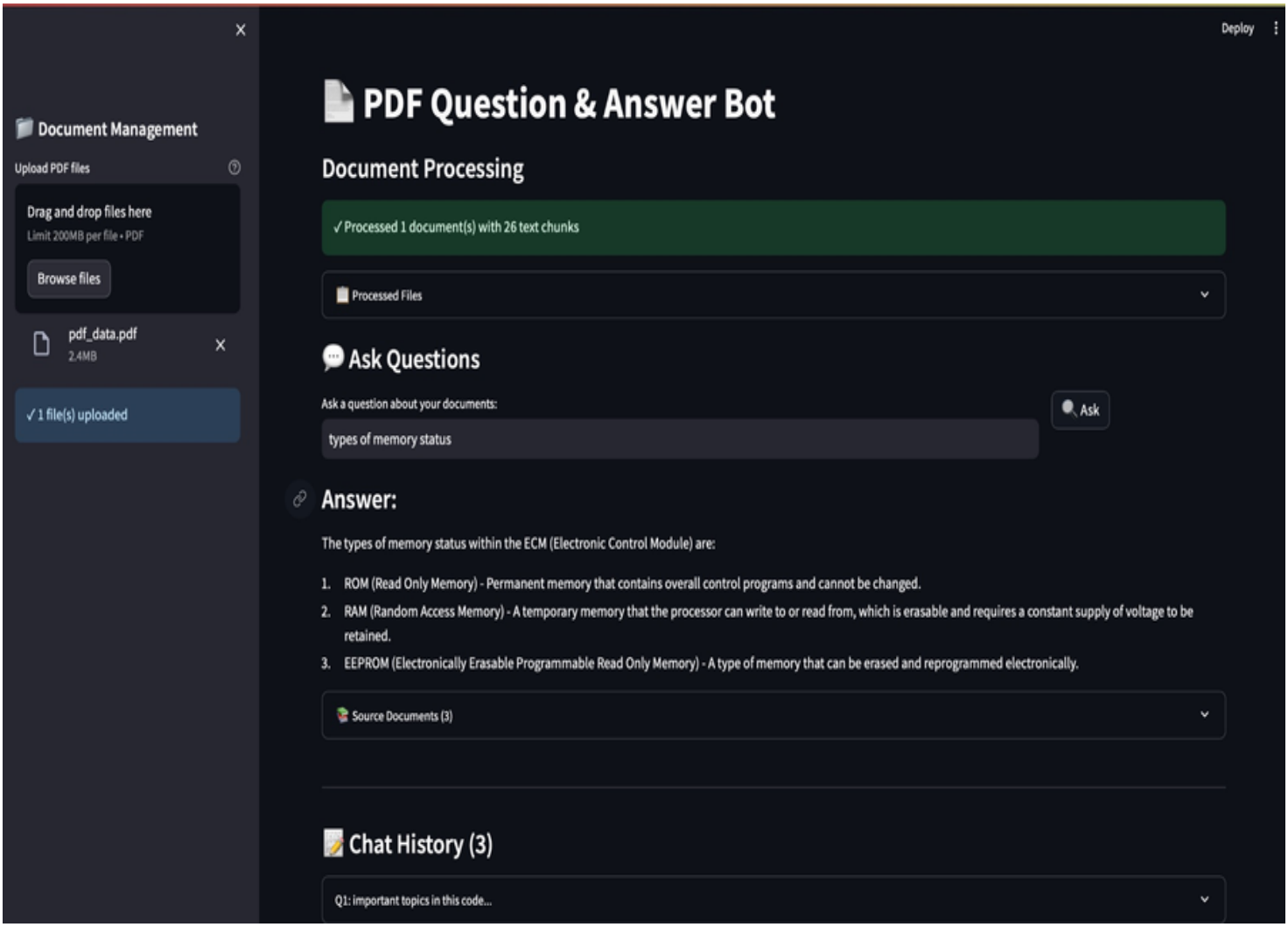


*Figure 1 — PDF Q&A Bot running on localhost with pdf_data.pdf loaded*

## 10.1 Sidebar — Document Management

The left panel handles all file management. It contains a drag-and-drop upload zone (limit 200 MB per file, PDF only), a Browse files button for manual selection, and a live status indicator that confirms how many files are currently loaded. In the screenshot, **pdf_data.pdf (2.4 MB)** has been uploaded and the confirmation badge reads "✓ 1 file(s) uploaded".

## 10.2 Document Processing Status

After upload, the main area immediately shows a green success banner confirming how many documents were processed and how many text chunks were created. In this example, the single PDF produced **26 text chunks**

which are stored in the FAISS vector index. The collapsible "Processed Files" expander lists each file by name for verification.

## 10.3 Ask Questions Panel

The question input area sits below the processing status. Users type a natural-language question into the wide text field and click the **Ask** button. In the screenshot, the query "types of memory status" has been submitted. Both the input and the button are automatically disabled when the hourly rate limit is reached.

## 10.4 Answer Display

The answer is rendered directly below the input in rich Markdown format. The model correctly identified and listed the three types of memory in an ECM module (ROM, RAM, EEPROM) with concise definitions, drawn from the retrieved document chunks. Below the answer, a collapsible **Source Documents (3)** expander shows the exact PDF passages the model used, giving full traceability back to the source material.

## 10.5 Chat History

At the bottom of the page, all questions asked during the session are preserved in the Chat History section. The count in the heading ("Chat History (3)") updates in real time. Each entry is a collapsible expander labelled with a truncated version of the question, showing the full Q&A pair when expanded. This history is also fed back into the LangChain chain on every new question, enabling natural follow-up queries that reference earlier answers.